

RISC vs. CISC: Instruction Set Design Principles & Trade-offs

ISA boundaries, microarchitectural costs, and quantitative trade-offs

Dr. J. Burns

Computer Systems Architecture — AArch64 Reference Platform

Topic 2: Module Topics

① ISA Foundations & The Semantic Trade-off Space

- The programmer-visible hardware contract and complexity redistribution.
- Granularity, memory access semantics, and early historical constraints.

② The CISC Architectural Model

- Register-memory operations, variable-length encodings, advantages, and decode costs.

③ The RISC Revolution & Principles

- Workload findings, load/store discipline, fixed 32-bit width, and compiler collaboration.
- Comparative execution breakdown: AArch64 vs. x86-64.

④ Quantitative Performance, Energy & Modern Convergence

- Iron Law evaluation, code density, pipeline regularity, and power/thermal constraints.
- Modern x86 μ op decomposition, AArch64 scaling from Apple Silicon to cloud silicon.

PART 1

ISA Foundations & Historical Context

What Is an Instruction Set Architecture?

- An ISA is the **programmer-visible functional specification** of a processor.
- It defines instructions, register files, data addressing modes, and architectural state.
- Compilers generate code targeting the ISA; processors implement the execution engine.
- Sits directly at the hardware/software boundary as a long-lived abstraction contract.
- **Primary Design Question:** Which operations belong in hardware primitives versus software composition?



Why ISA Design Matters

Software Consequences

- Dynamic instruction count (*IC*)
- Static binary footprint and code density
- Compiler optimization complexity
- ABI and register calling conventions

Hardware Consequences

- Parallel fetch and decode complexity
- Pipeline stage balance and depth
- Register port counts and ALU datapath
- Energy consumed per decoded instruction

Complexity Distribution Principle

ISA design does not eliminate system complexity; it **redistributes** complexity between software (compilers) and hardware (microarchitectures).

The Central ISA Design Questions

- **Operation Granularity:** Should one instruction perform a single primitive step or an aggregated multi-step sequence?
- **Memory Access Semantics:** Should arithmetic instructions accept memory operands directly, or strictly enforce a **Load/Store** boundary?
- **Encoding Uniformity:** Should instructions have fixed length (e.g., 32-bit words) or variable lengths (1–15 bytes)?
- **Addressing Complexity:** How many complex addressing modes (scaled indices, indirect pointers) should the hardware datapath evaluate?

Historical Constraints Shaped Early ISAs (CISC Era)

- **Expensive, Slow Memory (1960s–1970s):** Magnetic core and early DRAM had small capacity and high latency; minimizing binary size was critical.
- **Assembly-Centric Programming:** Programs were largely hand-written; rich instructions helped programmers express high-level constructs directly.
- **The Semantic Gap:** Designers believed machine instructions should mirror high-level statements directly (e.g., string manipulation in hardware).
- **Microprogrammed ROM Control:** Complex multi-cycle instructions were cheap to implement by executing micro-routines from on-chip ROM.

The CISC Architectural Model

The Origins of CISC

- **CISC:** *Complex Instruction Set Computer*.
- Emphasized rich instruction repertoires and diverse addressing modes.
- Permitted complex, multi-step actions within a single instruction (e.g., memory-indirect arithmetic).
- **Variable-Length Encoding:** Frequently used instructions occupied 1 byte; complex variants scaled up to 15 bytes.
- **Historical Implementations:** VAX-11, Motorola 68000, and the x86 architecture.

The CISC Design Philosophy

- **High Expressiveness:** Provide specialized instructions so software requires fewer total instructions to describe a task.
- **Flexible Operands:** Permit memory operands directly in ALU operations (e.g., `add [rdi], rax`).
- **Complex Addressing Modes:** Compute complex effective addresses ($\text{Base} + \text{Index} \times \text{Scale} + \text{Disp}$) within standard instructions.
- **Compact Encoding:** Maximize code density to economize on memory footprint and instruction fetch bandwidth.
- **Implementation Cost:** Accepts multi-stage decoders and irregular execution latencies in exchange for semantic density.

CISC in Practice: Register-Memory Model

CISC Instruction Example (x86-64)

```
add [rdi + 8], rax
```

Architectural View (Software)

- 1 instruction fetched.
- Compact variable encoding (3–7 bytes).
- Single atomic arithmetic operation directly to memory.

Microarchitectural Steps (Hardware)

- 1 Compute address: $EA = rdi + 8$.
- 2 Load memory contents at EA into ALU.
- 3 Add contents of register `rax`.
- 4 Store result back to memory address EA.

One architectural instruction decomposes into four discrete hardware operations.

Advantages of the CISC Approach

- **High Code Density:** Variable-length instructions minimize binary sizes, maximizing instruction-cache (I-cache) hit rates.
- **Reduced Dynamic Instruction Count (IC):** Fewer instructions fetched per loop or function invocation.
- **Direct Hardware Operations:** Dedicated instructions for string manipulation, polynomial evaluations, and memory-locked atomic sequences.
- **Ecosystem Compatibility:** Preserves backward binary compatibility across processor generations without recompilation.

The Microarchitectural Cost of CISC

- **Variable-Length Fetch Bottleneck:** Hardware cannot locate the start of instruction $N + 1$ until instruction N is fully decoded.
- **Decode Complexity:** Instruction decoders require multiple pipeline stages, consuming substantial silicon area and dynamic power.
- **Irregular Execution Latencies:** Instructions range from 1 cycle to tens of cycles, complicating pipeline hazard detection and register forwarding.
- **Exception Handling Complexity:** Instructions that modify multiple registers or memory locations make precise interrupt recovery difficult.

PART 3

The RISC Philosophy & Principles

The 1980s Shift: Emergence of RISC

- **Empirical Workload Analysis:** Patterson & Hennessy demonstrated that compiled code used only a small, simple subset of CISC instructions > 90% of the time.
- **Semiconductor Advancements:** On-die SRAM enabled larger register files (32+ registers), reducing memory traffic.
- **Pipelined Execution:** Overlapped pipeline stages heavily penalized variable-latency, irregular instructions.
- **Compiler Advancements:** Modern compilers optimize fine-grained primitive operations more effectively than monolithic instructions.

The RISC Design Philosophy

- **Uniform Fixed-Length Encodings:** Instructions are uniformly 32 bits, simplifying parallel instruction fetch and decode.
- **Strict Load/Store Architecture:** ALU operations use *only* registers; memory is accessed via explicit `ldr/str`.
- **Large Register File:** 32 general-purpose registers keep active variables in hardware.
- **Single-Cycle Execution:** Target uniform latency per pipeline stage to prevent pipeline bubbles.
- **Compiler-Exposed Primitives:** Direct hardware building blocks exposed for compiler scheduling.

Foundational RISC Lineages

- **Berkeley RISC (I/II):** Pioneered simplified instruction formats and register windows; direct precursor to SPARC.
- **Stanford MIPS:** Emphasized clean pipeline integration and compiler-managed hazards without microcode control.
- **Acorn RISC Machine (ARM, 1985):** Designed for low-power, high-efficiency client computing; evolved into modern **AArch64**.
- **Empirical Finding:** Processors executing simple instructions with lower CPI and high clock frequency consistently outperformed complex architectures.

RISC Principle: Simple and Regular Encodings

- Fixed 32-bit instructions align naturally with 4-byte, 16-byte, and 64-byte memory boundaries.
- Register specifiers and opcodes reside in fixed bit positions across instruction formats.
- Hardware decodes register specifiers *in parallel* with opcode evaluation.
- Eliminates the multi-stage sequential decode engines required by variable-length CISC.

RISC Principle: Load/Store Architecture

- Arithmetic and logical operations **never** accept direct memory operands.
- Decouples data transfer from arithmetic processing:
 - ① **Load:** Transfer data from DRAM/cache into register file.
 - ② **Compute:** Perform arithmetic across register operands.
 - ③ **Store:** Write computed results back to DRAM/cache.
- Eliminates pipeline stalls caused by cache misses occurring mid-ALU computation.
- **AArch64 Specification:** Strictly follows load/store execution rules.

RISC Principle: Register-Centric Computation

- Register access latency is single-cycle (~ 0.3 ns); DRAM latency is significantly higher (50–100 ns).
- 31 general-purpose 64-bit registers in AArch64 (x0–x30) keep inner-loop datasets in architectural state.
- Register-to-register operations eliminate bus traffic and memory port contention.
- **Compiler Responsibility:** Global graph-coloring algorithms allocate variables to registers to minimize memory spills.

RISC Principle: Fixed 32-Bit Instruction Width

AArch64 Fixed-Width Fetch

- Every instruction = 4 bytes.
- A 16-byte fetch window contains **exactly 4 instructions**.
- 4 parallel decoders operate concurrently without dependency chains.

CISC Variable-Length Fetch

- Instruction size spans 1 to 15 bytes.
- Hardware cannot determine the boundary of instruction 2 until instruction 1 is parsed.
- Requires pre-decode stages and multiplexing hardware.

RISC Principle: Compiler-Friendly ISA

- Exposes fine-grained, orthogonal primitives rather than composite operations.
- Enables aggressive compiler optimization phases:
 - **Common Subexpression Elimination (CSE):** Evaluate redundant expressions once into a register.
 - **Loop Invariant Code Motion:** Hoist invariant loads out of critical loops.
 - **Instruction Scheduling:** Reorder independent operations to fill pipeline latency slots.
- A regular ISA provides predictable analytical models for compiler cost analysis.

Code Example: Memory Addition in RISC

High-level expression: $c = a + b$; (where operands reside in memory)

AArch64 Assembly (Explicit Load/Store)

<code>ldr x1, [x0, #0]</code>	load a from memory into register $x1$
<code>ldr x2, [x0, #8]</code>	load b from memory into register $x2$
<code>add x3, x1, x2</code>	perform addition in register file
<code>str x3, [x0, #16]</code>	store sum c back to memory

- 4 discrete instructions, each performing one operational step.
- Fixed 32-bit width: Program footprint is exactly $4 \times 4 = 16$ bytes.
- Compilers can schedule independent instructions between `ldr` and `add` to hide memory latency.

Code Example: Memory Addition in CISC

High-level expression: $c = a + b$;

x86-64 CISC Assembly

<code>mov rax, [rdi]</code>	load <i>a</i> into rax
<code>add rax, [rdi + 8]</code>	read <i>b</i> from memory and add directly to rax
<code>mov [rdi + 16], rax</code>	store sum <i>c</i> back to memory

- 3 instructions instead of 4 (the add instruction performs both memory read and addition).
- Variable-length encoding (3 to 7 bytes per instruction).
- Hardware decoders decompose `add rax, [rdi+8]` into two internal operations: a memory load and an ALU operation.

Architectural vs. Microarchitectural Work

Metric	AArch64 (RISC)	x86-64 (CISC)
Architectural Instructions	4 instructions	3 instructions
Memory Accesses	2 loads, 1 store	2 loads, 1 store
ALU Operations	1 addition	1 addition
Decoded Micro-operations (μ ops)	4 operations	4 operations
Total Byte Footprint	16 bytes (fixed)	$\sim$ 13 bytes (variable)

Hardware Observation

The CISC program contains fewer architectural instructions, but the underlying arithmetic and memory operations executed by hardware are identical.

Quantitative Performance & Trade-offs

Instruction Count: Evaluation

- CISC reduces dynamic instruction count (IC) by combining multiple operations into single instructions.
- However, execution time is determined by total clock cycles, not just instruction count.
- A composite instruction performing three operational steps typically requires three or more clock cycles to complete.
- A 30% reduction in IC accompanied by a 60% increase in CPI results in a net performance loss.

Cycles Per Instruction (CPI) in RISC vs. CISC

- **RISC Target:** Aim for $CPI \approx 1.0$ (or $IPC > 1$ on superscalar processors) by ensuring instructions are uniform.
- **CISC Complexity:** Multi-step instructions occupy pipeline stages longer, increasing average CPI.
- Modern superscalar processors issue 4–8 instructions per cycle, favoring instructions with uniform latencies.
- Memory stalls impact both architectures, but decoupled RISC instructions allow cleaner pipeline latency hiding.

Clock Cycle Time (T_{clk}) and Critical Paths

- Clock frequency ($f_{\text{clk}} = 1/T_{\text{clk}}$) is constrained by the longest combinational path in any pipeline stage.
- Variable-length instruction parsing adds gate delays to front-end decode stages.
- Adding pipeline stages to maintain high frequency increases branch misprediction penalties.
- Simpler RISC decode stages allow shorter front-end pipelines at high clock frequencies.

The Performance Equation (Iron Law)

Processor Execution Time Equation

$$T_{\text{CPU}} = IC \times CPI \times T_{\text{clk}} = \frac{IC \times CPI}{f_{\text{clk}}}$$

- **CISC Strategy:** Minimize instruction count (IC) at the expense of higher CPI and decoder complexity.
- **RISC Strategy:** Minimize CPI and T_{clk} through regularity, accepting higher IC .
- **Trade-off Rule:** An architectural modification is beneficial only if the product of all three terms decreases.

Worked Performance Comparison

Workload: High-performance data-processing kernel.

Benchmark Workload

Design	IC	CPI	f_{clk}
AArch64	1.4×10^9	1.10	3.0 GHz
x86-64	1.0×10^9	1.60	3.0 GHz

Execution Time Calculations

$$T_{\text{AArch64}} = \frac{1.4 \times 10^9 \times 1.10}{3.0 \times 10^9} = \mathbf{0.513 \text{ s}}$$

$$T_{\text{x86-64}} = \frac{1.0 \times 10^9 \times 1.60}{3.0 \times 10^9} = \mathbf{0.533 \text{ s}}$$

Performance Result

The RISC machine executes 40% more instructions, but completes the workload faster because its lower CPI (1.10 vs. 1.60) compensates for the instruction count.

Code Density and Memory Footprint

- **Code Density:** The quantity of executable work expressed per byte of program binary.
- Variable-length CISC achieves high code density (e.g., 1-byte increment and return instructions).
- Fixed 32-bit RISC instructions consume more binary memory for simple operations.
- **Cache Implications:** Higher code density packs more instructions into the L1 instruction cache, reducing miss rates.
- **Compressed Encodings:** Modern RISC designs provide 16-bit compressed extensions (e.g., ARM Thumb-2, RISC-V 'C') to match CISC code density.

Instruction Fetch & Decode Complexity

RISC Fixed 32-Bit Decode

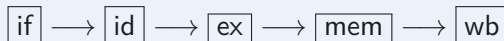
- Direct alignment: Instruction k is located at $pc + 4k$.
- Parallel decode across wide execution engines (e.g., 8-wide decode).
- Low silicon area and power consumption in the processor front end.

CISC Variable-Length Decode

- Instruction lengths vary from 1 to 15 bytes.
- Requires multi-stage pre-decoders to identify instruction boundaries.
- Higher front-end silicon area; modern x86 cores cache decoded μops (DSB).

ISA Design and Pipeline Efficiency

Classic 5-Stage RISC Pipeline



- Pipelining relies on dividing execution into balanced, synchronized stages.
- In RISC, instructions pass through these stages in order without skipping or looping.
- CISC instructions with memory-arithmetic operations require multiple execution or memory passes, causing pipeline stalls.

ISA Design and Modern Compiler Optimization

- Modern optimizing compilers (LLVM, GCC) maintain intermediate representations (IR) that resemble simplified RISC instructions.
- Mapping IR to regular RISC assembly is direct and exposes instruction-level parallelism (ILP).
- Complex CISC instructions create optimization ambiguities: compilers must evaluate whether compound instructions are faster than sequences of primitives.
- The RISC division of labor allows compilers to perform register allocation and scheduling globally.

Energy and Thermal Trade-offs

Front-end Energy Dissipation

- Fetching and decoding instructions consumes 15–30% of core power.
- Simple decoders reduce energy consumption per decoded instruction.

DRAM & Cache Energy

- Accessing off-chip DRAM consumes orders of magnitude more energy than on-chip logic.
- Denser code reduces memory bus traffic and instruction cache misses.

Efficiency Rule

Energy efficiency is determined by how effectively the microarchitecture minimizes front-end decode overhead and off-chip memory traffic.

Workload Energy: Energy per Program

Workload Energy Equation

$$E_{\text{program}} = P_{\text{avg}} \times T_{\text{CPU}} = IC \times E_{\text{instruction}}$$

- Lower energy per instruction ($E_{\text{instruction}}$) is beneficial only if total instruction count does not increase disproportionately.
- Finishing execution faster allows the processor to enter low-power sleep states sooner.
- Energy efficiency must be evaluated as **total Joules consumed per complete workload**.

Power Dissipation and Thermal Throttling

Dynamic Power Equation

$$P = C \cdot V_{\text{dd}}^2 \cdot f + P_{\text{static}}$$

- Complex decoding logic increases active switching capacitance (C).
- Higher clock frequencies (f) require higher supply voltages (V_{dd}).

Thermal Design Limits

Power $\longrightarrow$ Heat $\longrightarrow$ Throttling

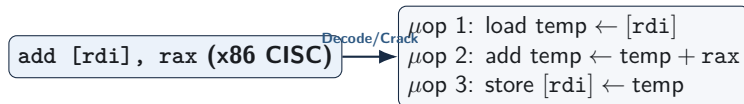
- Processors operate within a maximum Thermal Design Power (TDP).
- Thermal saturation forces dynamic frequency scaling, reducing sustained throughput.

Performance per Watt: Primary Evaluation Metric

- Mobile systems are constrained by battery capacity (5–10 W).
- Cloud data centers are constrained by rack power limits and cooling infrastructure (50–100 kW per rack).
- Performance-per-Watt ($\frac{\text{Workload Throughput}}{\text{Watts}}$) is the primary metric for processor design.
- AArch64's streamlined front end enables high IPC at low decode power, yielding high energy efficiency.

Microarchitectural Convergence: Modern x86 Internals

Modern x86 processors preserve the CISC ISA externally, but execute a **RISC datapath internally**:



- The execution core is an out-of-order execution engine processing micro-operations (μ ops).
- The CISC-to-RISC translation occurs dynamically in hardware decoders.

Complexity in Modern RISC ISAs

- “Reduced” refers to **instruction regularity**, not a constrained feature set.
- Modern AArch64 includes extensive extensions:
 - SIMD / NEON and Scalable Vector Extensions (SVE/SVE2).
 - Hardware floating-point, cryptographic acceleration, and virtualization.
 - Memory consistency barriers, acquire/release atomics, and Pointer Authentication (PAC).
- AArch64 microarchitectures employ wide out-of-order pipelines while preserving fixed-length instruction decoding.

RISC in Client and Desktop Computing

- Modern AArch64 implementations scale into high-performance desktop and workstation computing.
- **Architectural Factors:**
 - Fixed 32-bit instructions make wide decoders (8 wide) feasible at lower power budgets.
 - Large out-of-order execution windows achieve high IPC at moderate clock frequencies.
- Provides sustained computational performance within fanless and mobile thermal envelopes.

RISC in Data Center Infrastructure

- Cloud providers deploy custom AArch64 processors for server workloads:
 - **Google Axion:** Custom AArch64 CPUs for infrastructure and analytics.
 - **AWS Graviton:** Scaled-out cloud instances.
 - **Ampere Altra / AmpereOne:** High-density server platforms.
- Targeted for distributed databases, stream processing, and web tiers.
- High performance-per-watt maximizes compute density per server rack.

Data Center Workload Classification

Databases & Storage

- High-throughput Key-Value stores
- Distributed SQL engines
- In-memory caching tiers

Web Serving & Services

- High-concurrency reverse proxies
- Containerized microservices
- Indexing and ad-serving engines

Analytics & Pipelines

- Batch computation pipelines
- Stream processing frameworks
- Query execution engines

CPU AI & Preprocessing

- Tokenization and embedding lookup
- CPU-based inference workloads
- Feature normalization pipelines

AArch64 Reference Architecture Overview

- **Orthogonal Registers:** 31 general-purpose 64-bit registers (x0–x30), Zero Register (xzr), and Stack Pointer (sp).
- **Uniform 32-bit Encodings:** Regular instruction formats simplify hardware decoding.
- **Strict Load/Store Semantics:** Memory access occurs exclusively via `ldr` and `str`.
- **Platform Scope:** Standard architecture spanning mobile devices, client systems, servers, and supercomputers.

Modern ISA Comparison Summary

- The historical RISC vs. CISC distinction has evolved into a continuum of microarchitectural trade-offs.
- CISC processors translate complex instructions into internal RISC μ ops.
- RISC architectures incorporate advanced extensions for vector math and security.
- Fixed-length, load/store ISAs reduce front-end decode overhead and simplify wide execution pipelines.
- Hardware selection requires quantitative evaluation across instruction count, CPI, clock frequency, and memory bandwidth.

Summary: Evaluating Architectural Trade-offs

- **ISA Contract:** Determines the boundary and division of labor between compiler and hardware.
- **Design Trade-offs:** Dense instructions increase decode complexity; simple instructions increase instruction count.
- **Quantitative Framework:** Evaluate architectures using the Performance Equation ($T_{\text{CPU}} = \frac{IC \times CPI}{f_{\text{clk}}}$) and Performance-per-Watt.

Evaluation Framework

Rather than evaluating which architecture is inherently superior, quantify what trade-offs an ISA introduces and how the microarchitecture balances them for a given workload.

Review and Self-Test Questions

Topic 2: Comprehensive Summary

- **ISA Interface Contract:** The ISA redistributes complexity between compiler and hardware; it dictates instruction granularity, memory model, and encoding rules.
- **CISC Heritage:** Originated under expensive memory constraints; leverages variable-length encoding and register-memory operations to maximize code density.
- **RISC Philosophy:** Emphasizes fixed 32-bit instructions, load/store separation, large orthogonal register sets, and single-cycle pipeline friendliness.
- **Microarchitectural Convergence:** Modern CISC processors decode complex instructions into internal RISC-like μ ops in hardware, incurring a front-end decode and power penalty.
- **Quantitative Figure of Merit:** System performance and power must be evaluated via the Iron Law ($T_{\text{CPU}} = \frac{IC \times CPI}{f_{\text{clk}}}$) and Performance-per-Watt across complete workloads.
- **Forward Transition:** Topic 3 explores the concrete AArch64 programmer's model, register conventions, and basic instruction execution

Self-Test Questions: ISA Principles & Mechanics

- ❶ **Load/Store Boundary:** Why does a strict load/store architecture (such as AArch64) simplify pipelined execution compared to an architecture that allows direct memory operands in arithmetic instructions?
- ❷ **Parallel Instruction Fetch:** Explain why an 8-wide parallel instruction decoder is significantly simpler and more power-efficient to build for a fixed 32-bit RISC ISA than for a variable-length CISC ISA.
- ❸ **Micro-operation Decomposition:** When an x86 processor executes `add [rdi + 8], rax`, how does the front-end decoder translate this single architectural instruction into internal micro-operations (μ ops)?
- ❹ **Code Density Trade-off:** Under what system constraints (e.g., cache sizes, memory bandwidth) does the superior code density of a CISC ISA translate to an execution performance advantage?

Self-Test Questions: Quantitative Analysis & Power

- ❶ **Iron Law Trade-off Calculation:** A benchmark compiled for AArch64 executes 1.5×10^9 instructions with an average CPI of 1.1 on a 3.0 GHz CPU. The same benchmark compiled for x86-64 executes 1.1×10^9 instructions with an average CPI of 1.6 on a 3.2 GHz CPU. Which architecture completes the task faster, and what is the speedup?
- ❷ **Frontend Power Budget:** Why does front-end instruction decoding account for a higher percentage of core power in modern x86 processors than in modern AArch64 cores?
- ❸ **Performance-per-Watt at Scale:** In cloud data centers, why has Performance-per-Watt largely superseded raw single-core clock frequency as the primary procurement and design metric?
- ❹ **Microarchitectural Misconceptions:** Why is it technically inaccurate to describe modern high-performance AArch64 cores (such as Apple M-series or Neoverse cores) as “simple processors”?